

Guia Técnico

Referência para manutenção e resolução de problemas

Este guia cobre a arquitetura do site, integração com o GitHub, configuração de acesso SSH, solução de erros comuns e procedimentos de manutenção.

Destinado a quem tem noção básica de programação ou linha de comando.

SUMÁRIO

ARQUITETURA

1. Visão geral do sistema	3
Estrutura de arquivos	3
Fluxo de dados	3
2. Scripts JavaScript	4
Ordem de carregamento	4
dataReady e Promise	4

GITHUB

3. GitHub Pages e repositório	5
Como o site é publicado	5
Personal Access Token (PAT)	5
4. Integração com GitHub API	6
Como o admin salva dados	6
Como o admin carrega dados	6

ACESSO E SEGURANÇA

5. Token e criptografia	7
Como o token é armazenado	7
Proteção contra brute force	7
6. SSH e Git local	8

RESOLUÇÃO DE PROBLEMAS

7. Erros comuns do admin	9
8. Erros do GitHub	10
9. Problemas de exibição	11
10. Manutenção e atualizações	12

REFERÊNCIA

11. data.json — estrutura completa	13
12. Variáveis CSS e temas	14
13. Checklist de deploy	15

SEÇÃO 1

Visão geral do sistema

Arquitetura, estrutura de arquivos e fluxo de dados

O site **pontos de fuga** é um site estático hospedado no **GitHub Pages**. Não existe servidor backend — todo o conteúdo dinâmico é gerenciado via um painel admin no próprio browser, que lê e escreve diretamente em um arquivo JSON no repositório usando a GitHub API.

Estrutura de arquivos

```
site-produtora/ └─ index.html ← página inicial └─ producoes.html ← catálogo de filmes └─ filme.html ← página individual de filme └─ historia.html ← sobre / time / festivais └─ vemai.html ← filmes em produção └─ contato.html ← formulário de contato └─ admin.html ← painel admin (não indexado) └─ robots.txt ← bloqueia /admin.html dos crawlers └─ sitemap.xml ← mapa do site para SEO └─ scripts/ └─ data.json ← TODA a base de dados do site └─ data.js ← carrega data.json, expõe dataReady └─ script-shared.js ← nav, menu, lang, footer – todas as páginas └─ script-index.js ← lógica exclusiva do index.html └─ script-filme.js ← lógica exclusiva do filme.html └─ script-producoes.js └─ script-historia.js └─ script-vemai.js └─ script-contato.js └─ script-admin.js ← painel admin └─ style/ └─ style-base.css ← variáveis, reset, nav, footer └─ style-admin.css └─ style-[página].css
```

Fluxo de dados

Leitura (ao abrir o site)

O browser carrega `data.js` → faz fetch de `data.json` → resolve a Promise `dataReady` → cada script de página renderiza o conteúdo.

Escrita (ao salvar no admin)

Admin coleta os campos → serializa para JSON → chama GitHub Contents API com PUT → GitHub atualiza o arquivo no repositório → GitHub Pages publica em ~1 min.

IMPORTANTE

O site não usa banco de dados nem servidor. Todos os dados vivem em `scripts/data.json` no GitHub. Qualquer edição feita pelo admin é um commit no repositório.

Scripts JavaScript

Ordem de carregamento, dependências e o padrão `dataReady`

Ordem de carregamento

Cada página HTML carrega os scripts nesta ordem, com `defer`:

EXEMPLO — INDEX.HTML

```
<script src="scripts/data.js" defer></script>
<script src="scripts/script-shared.js" defer></script>
<script src="scripts/script-index.js" defer></script>
```

O atributo `defer` garante que os scripts executam na ordem declarada, depois que o HTML foi parseado — mas **não** esperam o fetch de `data.json` terminar. Por isso existe o padrão `dataReady`.

`dataReady` — a Promise central

Em `data.js`, a variável global `dataReady` é uma Promise que resolve quando os dados JSON estão carregados e disponíveis nas variáveis globais:

SCRIPTS/DATA.JS (SIMPLIFICADO)

```
const dataReady = fetch('scripts/data.json')
  .then(r => r.json())
  .then(json => {
    films = json.films || [];
    upcomingFilms = json.upcomingFilms || [];
    otherProductions = json.otherProductions || [];
    historiaData = json.historia || {};
    pagesData = json.pages || {};
    siteData = json.siteData || {};
  });
```

Cada script de página envolve sua lógica em `dataReady.then(() => { ... })`. Isso garante que as variáveis estão prontas antes de tentar renderizar.

Variáveis globais disponíveis após `dataReady`

VARIÁVEL	TIPO	CONTEÚDO
<code>films</code>	Array	Filmes do catálogo principal
<code>upcomingFilms</code>	Array	Filmes em produção ("vem aí")
<code>otherProductions</code>	Array	Outras produções
<code>historiaData</code>	Object	Manifesto, time, marcos, parceiros, festivais

`pagesData`

Object

Conteúdo editável de cada página (index, vemai, etc.)

`siteData`

Object

Logo, links sociais, configurações gerais

window.updateDOM — troca de idioma

Cada script de página define `window.updateDOM = function() {...}`. Quando o usuário troca o idioma (PT/EN), `script-shared.js` chama essa função para re-renderizar todo o conteúdo dinâmico na nova língua.

ATENÇÃO

Se você adicionar um novo campo de texto ao JSON e renderizá-lo via JS, lembre de incluir a re-renderização dentro de `window.updateDOM`, senão o texto não atualiza ao trocar idioma.

GitHub Pages e repositório

Como o site é hospedado e o que é necessário para funcionar

Como o site é publicado

O site está no repositório `myceliumBrain/site-produtora`, branch `lite_mode`. O GitHub Pages está configurado para servir os arquivos desta branch diretamente na raiz.

URL do site

`myceliumbrain.github.io/site-produtora`

Branch publicada

`lite_mode` (branch principal do projeto)

Para verificar ou alterar a configuração: **GitHub** → **repositório** → **Settings** → **Pages** → **Source**.

Personal Access Token (PAT)

O painel admin precisa de um **Personal Access Token** do GitHub para conseguir salvar dados. Sem ele, o botão "Salvar" retorna erro 401.

Como criar um novo token

- 1 Acesse `github.com` com sua conta e vá em **Settings** (foto de perfil → Settings).
- 2 Role até o fim do menu lateral e clique em **Developer settings**.
- 3 Clique em **Personal access tokens** → **Tokens (classic)**.
- 4 Clique em **Generate new token (classic)**.
- 5 Em **Note**, dê um nome como "admin site produtora".
- 6 Em **Expiration**, escolha "No expiration" (ou defina uma data se preferir mais segurança).
- 7 Em **Scopes**, marque apenas `repo` (dá acesso de leitura e escrita ao repositório).
- 8 Clique em **Generate token** e **copie o token imediatamente** — ele só aparece uma vez.
- 9 Cole o token no campo correspondente no painel admin ao fazer login.

SEGURANÇA

O token dá acesso de escrita ao repositório. Nunca compartilhe nem deixe salvo em lugar inseguro. Se vaziar, revogue imediatamente em GitHub → Developer settings → Personal access tokens.

Permissões necessárias do token

SCOPE	NECESSÁRIO?	PARA QUÊ
<code>repo</code>	Sim	Ler e escrever arquivos no repositório
<code>workflow</code>	Não	Não usado

admin:repo_hook

Não

Não usado

Integração com GitHub API

Como o admin lê e escreve dados no repositório

Como o admin carrega dados

Ao entrar no painel, o admin faz um `GET` para a GitHub Contents API:

REQUISIÇÃO GET

```
GET https://api.github.com/repos/myceliumBrain/site-produtora/contents/scripts/data.json
Authorization: token ghp_XXXXXXXXXXXX
Accept: application/vnd.github.v3+json
```

A resposta contém o campo `content` (conteúdo do arquivo em Base64) e o campo `sha` (hash do arquivo — necessário para fazer o PUT).

O admin decodifica o Base64 com `TextDecoder` e faz parse do JSON resultante.

Como o admin salva dados

Ao clicar em "Salvar", o admin faz um `PUT` para a mesma URL com o novo conteúdo:

REQUISIÇÃO PUT

```
PUT https://api.github.com/repos/myceliumBrain/site-produtora/contents/scripts/data.json
Authorization: token ghp_XXXXXXXXXXXX

{
  "message": "admin: atualiza data.json",
  "content": "<base64 do novo JSON>",
  "sha": "<sha do arquivo atual>"
}
```

O campo `sha` é obrigatório para evitar conflitos — o GitHub rejeita o PUT se o `sha` não corresponder à versão atual do arquivo.

CONFLITO DE SHA

Se duas abas do admin tentarem salvar ao mesmo tempo, a segunda irá falhar com erro 409 (conflito). Solução: recarregar a página admin e refazer as edições perdidas.

Verificando chamadas à API manualmente

Você pode testar a API diretamente no terminal para verificar se o token funciona e se o arquivo está acessível:

TERMINAL (BASH/ZSH)

```
curl -H "Authorization: token SEU_TOKEN_AQUI" \
-H "Accept: application/vnd.github.v3+json" \
```

```
https://api.github.com/repos/myceliumBrain/site-produtora/contents/scripts/data.json
```

Se retornar `"message": "Bad credentials"`, o token está errado ou expirou. Se retornar `"message": "Not Found"`, o repositório ou o caminho do arquivo estão errados.

Rate limit da API

AUTENTICADO	NÃO AUTENTICADO	VERIFICAR LIMITE
5.000 req/hora	60 req/hora	<code>GET /rate_limit</code>

Para uso normal do admin, o rate limit nunca será atingido.

Token e criptografia

Como o token é armazenado com segurança e a proteção contra ataques

Como o token é armazenado

O token do GitHub não é armazenado em texto puro. O admin usa **AES-GCM** com chave derivada via **PBKDF2** a partir da senha escolhida pelo usuário. O resultado criptografado é salvo no `localStorage` do browser.

Fluxo de criptografia

Senha + salt → PBKDF2 (100.000 iterações) → chave AES-GCM → criptografa token → salva no localStorage

Para descriptografar: a senha é pedida novamente a cada sessão → mesma derivação PBKDF2 → descriptografa o token salvo → token fica apenas na memória (RAM) enquanto o admin está aberto.

Chaves no localStorage

CHAVE	CONTEÚDO
<code>pf_enc_token</code>	Token GitHub criptografado (AES-GCM, Base64)
<code>pf_salt</code>	Salt aleatório usado na derivação PBKDF2 (Base64)
<code>pf_login_attempts</code>	JSON com contador e timestamp para brute force

ESQUECEU A SENHA?

Se a senha for esquecida, abra o DevTools do browser (F12) → Console e execute:

```
localStorage.removeItem('pf_enc_token'); localStorage.removeItem('pf_salt');
```

 — depois recarregue a página e configure novamente com um novo token.

Proteção contra brute force

O admin limita tentativas de login:

Tentativas permitidas

5 tentativas antes do bloqueio

Duração do bloqueio

15 minutos automáticos

O estado do bloqueio é salvo em `localStorage` (chave `pf_login_attempts`). Para desbloquear manualmente em emergência:

CONSOLE DO BROWSER (F12)

```
localStorage.removeItem('pf_login_attempts');
```

Redefinir acesso completamente

Para apagar todas as credenciais salvas e recomeçar do zero:

CONSOLE DO BROWSER (F12)

```
localStorage.removeItem('pf_enc_token');  
localStorage.removeItem('pf_salt');  
localStorage.removeItem('pf_login_attempts');  
location.reload();
```

ATENÇÃO

O localStorage é por domínio e por browser. Se a equipe usa computadores diferentes, cada um precisa configurar as credenciais no seu próprio browser.

SSH e Git local

Para alterações diretas no repositório via terminal

Embora o dia a dia seja via painel admin, em alguns casos (mudança de layout, correção de bug, atualização de código) pode ser necessário editar os arquivos e enviar via Git.

Configurar SSH com GitHub

- 1 Verificar se já existe uma chave SSH: `ls ~/.ssh/id_ed25519.pub` — se existir, pule para o passo 3.
- 2 Gerar nova chave (troque o email pelo seu):
`ssh-keygen -t ed25519 -C "seu@email.com"` — pressione Enter em tudo para usar os padrões.
- 3 Copiar a chave pública: `cat ~/.ssh/id_ed25519.pub` — copie todo o texto exibido.
- 4 No GitHub: **Settings** → **SSH and GPG keys** → **New SSH key** → cole a chave e salve.
- 5 Testar a conexão: `ssh -T git@github.com` — deve aparecer "Hi username! You've successfully authenticated".

Clonar o repositório

TERMINAL

```
git clone git@github.com:myceliumBrain/site-produtora.git
cd site-produtora
git checkout lite_mode
```

Fluxo de trabalho básico

TERMINAL — CICLO EDITAR → COMMITAR → PUBLICAR

```
# 1. Atualizar com as mudanças mais recentes
git pull origin lite_mode

# 2. Editar os arquivos necessários (no editor de sua preferência)

# 3. Ver o que mudou
git status
git diff

# 4. Adicionar as mudanças
git add scripts/data.json          # ou git add -A para tudo

# 5. Commitar com mensagem descritiva
git commit -m "fix: corrige título do filme X"

# 6. Enviar para o GitHub (publica o site)
git push origin lite_mode
```

Se o admin salvou e depois você faz `git pull`, pode acontecer conflito em `data.json`. Sempre faça `git pull` antes de editar localmente, e comunique a equipe antes de fazer push.

Comandos úteis de diagnóstico

COMANDO	O QUE FAZ
<code>git log --oneline -10</code>	Últimos 10 commits
<code>git diff HEAD~1 scripts/data.json</code>	O que mudou no data.json no último commit
<code>git show HEAD:scripts/data.json</code>	Conteúdo do data.json no último commit
<code>git stash</code>	Salva mudanças locais temporariamente
<code>git stash pop</code>	Restaura mudanças salvas com stash

Erros comuns do admin

Mensagens de erro, causas e soluções

ERRO "Senha incorreta" ao fazer login

A senha digitada não descriptografou o token. Pode ser erro de digitação (maiúsculas, espaço extra) ou o token foi configurado com senha diferente.

Solução: verifique caps lock. Se a senha realmente foi esquecida, limpe o localStorage (veja Seção 5) e reconfigure.

ERRO "Conta bloqueada por 15 minutos"

5 tentativas de senha erradas foram registradas.

Solução: aguardar 15 min OU abrir DevTools (F12) → Console → `localStorage.removeItem('pf_login_attempts');`
`location.reload();`

HTTP 401 Unauthorized — ao salvar

O token do GitHub está inválido, expirado, ou não tem a permissão `repo`.

Solução: gere um novo token (Seção 3) e reconfigure no admin. Lembre de marcar o scope `repo`.

HTTP 409 Conflict — ao salvar

O SHA do arquivo mudou desde que o admin carregou os dados. Outra pessoa salvou antes ou você tem duas abas do admin abertas.

Solução: recarregar o admin (F5), refazer as edições e tentar salvar novamente.

HTTP 403 Forbidden — ao salvar

O token não tem permissão de escrita no repositório. Pode ser um token fine-grained sem a permissão correta, ou o repositório está em organização com restrições.

Solução: use um token classic (não fine-grained) com scope `repo`.

ERRO "Erro ao carregar dados" — página em branco

O fetch de `data.json` falhou. Causas: arquivo corrompido (JSON inválido), falha de rede, ou o arquivo foi deletado do repositório.

Solução: verifique se `scripts/data.json` existe no repositório e se é um JSON válido (use `jsonlint.com` para validar).

ERRO Painel abre mas os campos aparecem vazios

Os dados carregaram mas uma seção específica está faltando no JSON (ex: `historia`, `pages`).

Solução: abrir o DevTools (F12) → Console e verificar se há erros JavaScript. Verificar a estrutura do `data.json` (Seção 11).

Erros do GitHub

Problemas com o repositório, Pages e publicação

Site não atualiza após salvar

O GitHub Pages pode demorar entre 30 segundos e 3 minutos para publicar depois de um commit. Verifique:

- 1 Confirme que o commit aparece no repositório: `github.com/myceliumBrain/site-produtora/commits/lite_mode`
- 2 Verifique o status do deploy: **repositório** → **Actions** (se houver workflow) ou **repositório** → **Environments** → **github-pages**.
- 3 Se o deploy falhou (ícone vermelho), clique no workflow para ver o log de erro.
- 4 Forçar atualização no browser: **Ctrl+Shift+R** (Windows/Linux) ou **Cmd+Shift+R** (Mac).

GitHub Pages não está ativo

- 1 No repositório, vá em **Settings** → **Pages**.
- 2 Em **Source**, selecione **Deploy from a branch**.
- 3 Em **Branch**, selecione `lite_mode` e pasta `/ (root)`.
- 4 Clique em **Save** e aguarde alguns minutos.

Recuperar versão anterior do data.json

Como cada salvamento é um commit, é possível voltar para uma versão anterior:

TERMINAL — VER HISTÓRICO DO DATA.JSON

```
# Ver commits que alteraram o data.json
git log --oneline scripts/data.json

# Ver o conteúdo em um commit específico (substitua HASH pelo hash do commit)
git show HASH:scripts/data.json

# Restaurar o data.json para uma versão anterior
git checkout HASH -- scripts/data.json
git commit -m "fix: restaura data.json para versão anterior"
git push origin lite_mode
```

BACKUP AUTOMÁTICO

O Git mantém todo o histórico de mudanças. Nada é perdido permanentemente — sempre é possível recuperar uma versão anterior de qualquer arquivo.

Erro de permissão ao fazer push

ERRO `Permission denied (publickey)` — ao fazer `git push`

A chave SSH não está configurada ou não foi adicionada ao GitHub.

Solução: seguir os passos da Seção 6 para configurar SSH. Verificar com: `ssh -T git@github.com`

ERRO `rejected — non-fast-forward`

O repositório remoto tem commits que você não tem localmente (alguém fez push antes de você).

Solução: `git pull --rebase origin lite_mode` e depois tente o push novamente.

Problemas de exibição

Conteúdo não aparece, imagens quebradas, texto errado

Imagem não aparece no site

Imagens são referenciadas por URL. Causas comuns de imagens quebradas:

CAUSA	COMO IDENTIFICAR	SOLUÇÃO
URL digitada errada	Imagem tem ícone de quebrado no browser	Verificar a URL no admin — abrir a URL diretamente no browser para testar
Imagem deletada da origem	URL retorna 404	Reupar a imagem e atualizar a URL
URL com espaços ou acentos	Funciona no desktop mas não no mobile	Usar URLs sem espaços (substituir por <code>%20</code> ou renomear o arquivo)
HTTP em vez de HTTPS	Erro de "mixed content" no console	Trocar <code>http://</code> por <code>https://</code> na URL

Texto aparece como HTML escapado (< & etc.)

Isso pode acontecer se o conteúdo for inserido em um campo que espera texto simples, mas você colou HTML. O site escapa automaticamente todos os caracteres especiais por segurança.

Solução: remova quaisquer tags HTML do texto no admin (ex: não use `<b>`, `<i>`). O site não suporta rich text.

Mudança de idioma não atualiza um campo

Cada campo bilíngue tem dois campos no JSON: um para PT e outro para EN (sufixo `En`). Se o campo EN estiver vazio, o site mostra o PT mesmo com idioma trocado. Verifique se o campo `titleEn`, `synopsisEn`, etc. está preenchido no admin.

Filme abre com "Filme não encontrado"

A URL de um filme usa um índice numérico (`?i=3`). Se filmes forem reordenados ou removidos, índices antigos ficam inválidos. Links no site são gerados automaticamente — mas links externos (redes sociais, etc.) podem ficar quebrados. Não há solução simples além de atualizar os links externos.

Console do browser — como abrir e ler erros

O console é a principal ferramenta de diagnóstico. Para abrir:

Windows / Linux

Mac

Depois

F12 ou Ctrl+Shift+I

Cmd+Option+I

Clique na aba **Console**

Erros em vermelho indicam problemas. Os mais comuns no site:

MENSAGEM NO CONSOLE	O QUE SIGNIFICA
Failed to fetch	Não conseguiu carregar <code>data.json</code> (sem internet ou arquivo não existe)
Unexpected token em JSON	<code>data.json</code> está com erro de sintaxe (vírgula extra, aspas faltando)
Cannot read properties of undefined	Um campo esperado está faltando no JSON
NetworkError	Problema de rede ou CORS ao acessar a API do GitHub

Manutenção e atualizações

Tarefas periódicas e procedimentos de atualização de código

Tarefas de manutenção periódica

TAREFA	FREQUÊNCIA	COMO FAZER
Renovar o token do GitHub	Se expirar	Seção 3 — gerar novo token e reconfigurá-lo no admin
Verificar links sociais	Semestral	Admin → Links — confirmar que os URLs ainda funcionam
Atualizar ano no sitemap.xml	Anual	Editar <code>sitemap.xml</code> no repositório e atualizar <code><lastmod></code>
Verificar imagens quebradas	Trimestral	Abrir cada página, inspecionar visualmente

Atualizar o código do site

Para mudanças de código (HTML, CSS, JS) que vão além do que o admin oferece:

- 1 Clone ou atualize o repositório local: `git pull origin lite_mode`
- 2 Faça as alterações nos arquivos necessários.
- 3 Teste localmente abrindo o HTML no browser (ou usando um servidor local: `python3 -m http.server 8000`).
- 4 Commit e push: `git add . && git commit -m "descricao" && git push origin lite_mode`
- 5 Aguarde 1-3 min para o GitHub Pages publicar e verifique o site.

Adicionar nova seção ou campo ao admin

O admin e os dados do site são acoplados. Se quiser adicionar um novo campo editável:

- 1 Adicionar o campo ao `data.json` com um valor padrão.
- 2 Expor o campo no script da página correspondente (`script-index.js` etc.) para renderização.
- 3 Adicionar o campo ao painel admin em `script-admin.js` (`renderAll()` + `collectAll()`).
- 4 Se o campo for bilíngue, adicionar versão PT e EN.

Estrutura de um filme no data.json

Para referência ao manipular diretamente:

DATA.JSON — OBJETO FILME

```
{
  "title":      "Título em PT",
  "titleEn":    "Title in EN",
```

```
"synopsis": "Sinopse...",
"synopsisEn": "Synopsis...",
"director": "Nome",
"genre": "Drama",
"genreEn": "Drama",
"year": "2025",
"imgPortrait": "https://...",
"imgLandscape": "https://...",
"hero": true, // aparece no slideshow da home
"tags": ["Drama", "2025"],
"videoTrailers": ["https://youtube.com/watch?v=..."],
"fichatecnica": [{"role": "Roteiro", "name": "Nome"}],
"elenco": [{"role": "Personagem", "name": "Atoz"}],
"fotografias": ["https://..."],
"makingOff": ["https://..."]
}
```


data.json — estrutura

Mapa completo das chaves do arquivo de dados

Estrutura de alto nível

DATA.JSON — RAIZ

```
{
  "films":          [...],    // Array de filmes do catálogo
  "upcomingFilms": [...],    // Array de filmes em produção
  "otherProductions": [...],  // Array de outras produções
  "historia":       {...},    // Conteúdo da página história
  "pages":          {...},    // Conteúdo editável de cada página
  "siteData":       {...}    // Config global (logo, links sociais)
}
```

historia

CHAVE	TIPO	DESCRIÇÃO
<code>manifesto</code>	Object	eyebrow, title1, title2, p1, p2 (e versões En)
<code>team</code>	Array	Membros: name, role, roleEn, bio, bioEn, img
<code>marcos</code>	Array	Marcos históricos: year, title, titleEn, text, textEn
<code>festivais</code>	Array	Festivais: name, year, logo
<code>parceiros</code>	Array	Parceiros: string OU {name, logo}

pages

CHAVE	DESCRIÇÃO
<code>pages.index</code>	ctaTitlePt/En, ctaBodyPt/En, heroLabelPt/En, gridTitlePt/En, stripItems
<code>pages.vemai</code>	eyebrowPt/En, titlePt/En, subPt/En, statementPt/En
<code>pages.historia</code>	teamEyebrowPt/En, festivaisEyebrowPt/En, marcosEyebrowPt/En, parceirosEyebrowPt/En

siteData

CHAVE	TIPO	DESCRIÇÃO
<code>logoUrl</code>	String	URL da imagem do logo

upcomingFilms — campos extras

Além dos campos padrão de filme, upcomingFilms tem:

CAMPO	VALORES	EXIBIÇÃO
status	"filming" / "dev" / "post"	Badge colorido na página vemai e home

JSON INVÁLIDO

Se `data.json` ficar com sintaxe inválida (edição manual errada), o site inteiro para de funcionar. Sempre valide o JSON antes de commitar: use `python3 -m json.tool data.json` no terminal ou o site jsonlint.com.

Variáveis CSS e temas

Como o design é controlado via custom properties

Todo o design visual do site é controlado por variáveis CSS definidas em `:root` no arquivo `style/style-base.css`. Para mudar cores, espaçamentos ou tipografia, basta alterar essas variáveis.

Variáveis de cor principais

VARIÁVEL	VALOR ATUAL	USO
<code>--black</code>	<code>#0a0a0a</code>	Fundo principal
<code>--dark</code>	<code>#111111</code>	Fundo de seções escuras
<code>--surface</code>	<code>#1a1a1a</code>	Cards e painéis
<code>--border-f</code>	<code>#222</code>	Bordas sutis
<code>--text</code>	<code>#d0cec9</code>	Texto principal
<code>--muted</code>	<code>#888</code>	Texto secundário
<code>--accent</code>	<code>#c8a96e</code>	Cor de destaque (dourado)

Variáveis de layout

VARIÁVEL	VALOR	USO
<code>--pad-desk</code>	<code>clamp(24px, 5vw, 80px)</code>	Padding lateral nas páginas
<code>--nav-h</code>	<code>64px</code>	Altura da barra de navegação

Tipografia

O site usa duas fontes do Google Fonts:

Fraunces

Fonte serifada para títulos e destaques. Carregada com pesos 300, 400, 600.

Space Grotesk

Fonte sans-serif para corpo de texto e interface. Pesos 300, 400, 500, 600.

Breakpoints responsivos

BREAKPOINT	VALOR	CONTEXTO
------------	-------	----------

Mobile	max-width: 768px	Layout de coluna única, nav mobile
Tablet	max-width: 1024px	Grid 2 colunas
Desktop	acima de 1024px	Layout completo

Acessibilidade — prefers-reduced-motion

O site respeita a preferência do sistema operacional por menos animação. Quando ativada (**Configurações** → **Acessibilidade** → **Reduzir movimento**), todas as durações de transição e animação são zeradas automaticamente via:

STYLE-BASE.CSS

```
@media (prefers-reduced-motion: reduce) {  
  *, *::before, *::after {  
    animation-duration: 0.01ms !important;  
    transition-duration: 0.01ms !important;  
  }  
}
```

Checklist de deploy

O que verificar antes e depois de publicar mudanças

Antes de fazer push

ITEM	COMO VERIFICAR
☐ data.json é JSON válido	<code>python3 -m json.tool scripts/data.json</code>
☐ Nenhuma imagem tem URL com HTTP (deve ser HTTPS)	Grep: <code>grep "http://" scripts/data.json</code>
☐ Abriu as páginas localmente sem erro no console	Servidor local: <code>python3 -m http.server 8000</code>
☐ Testou a troca de idioma (PT ↔ EN)	Clicar no botão de idioma em cada página
☐ Nenhum campo com texto HTML cru (ex: <code>texto</code>)	Verificar visualmente no site

Depois de fazer push

ITEM	COMO VERIFICAR
☐ Deploy concluído no GitHub Actions/Pages	GitHub → repositório → deployments
☐ Site atualizado no browser (hard refresh)	Ctrl+Shift+R
☐ Home carrega e slideshow funciona	Acessar index.html
☐ Página de filmes exibe catálogo	Acessar producoes.html
☐ Admin ainda consegue salvar	Editar e salvar algo sem erro

Referência rápida — comandos mais usados

TERMINAL — REFERÊNCIA RÁPIDA

```
# Servidor local para testes
python3 -m http.server 8000

# Validar JSON
python3 -m json.tool scripts/data.json

# Ver últimas mudanças
git log --oneline -10

# Buscar texto em todos os arquivos JS
grep -r "texto_buscado" scripts/

# Desfazer última mudança local (antes do commit)
git checkout -- scripts/data.json
```

```
# Ver diferença entre o que está local e o remoto
git fetch && git diff origin/lite_mode
```

Contatos e recursos

Repositório

github.com/myceliumBrain/site-produtora

GitHub Pages status

githubstatus.com — verificar se o serviço está no ar

Documentação GitHub API

docs.github.com/rest/repos/contents

Validador JSON

jsonlint.com — colar e validar o data.json

RESUMO DA ARQUITETURA

Site estático no GitHub Pages → dados em `data.json` no repositório → admin lê/escreve via GitHub API com token autenticado → cada salvamento = 1 commit → publicação automática em 1-3 min.